



Lab 1 - Model Context Protocol (MCP)

We'll make Copilot Chat's agent mode smarter using an MCP server.



Prerequisites

Ensure that you have NPM or Docker installed on your machine.



Exercise 1: Model Context Protocol integration

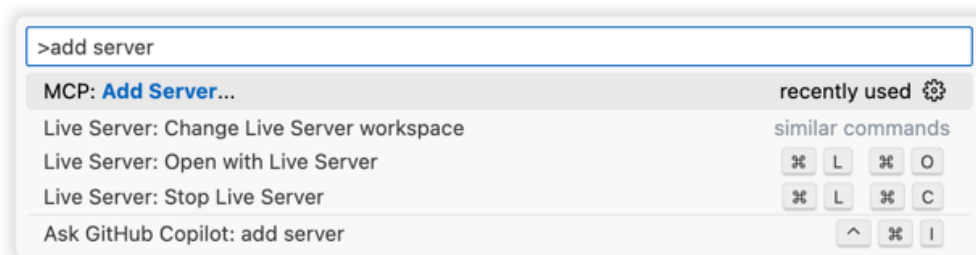
Objective

Use an MCP server to improve Copilot Chat's agent mode.

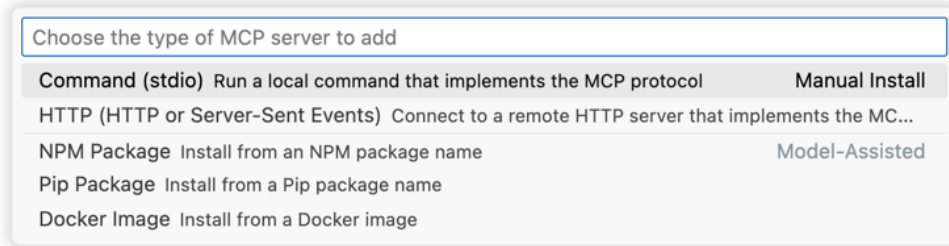
Instructions

Careful: MCP servers run arbitrary code on your machine. Only add servers from trusted sources, and review the publisher and server configuration before starting it.

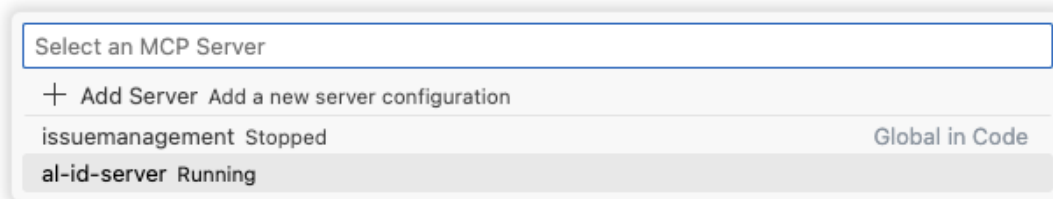
1. Go to <https://code.visualstudio.com/mcp> and find an MCP server that you are interested in.
2. Press the "Install" button.
3. From VS Code, press the "Start" button under the server's settings.
4. Fill in the required details to configure. You might have to authenticate with the server.
5. Optional: You can also find an MCP server outside the VS Code website, for example via <https://github.com/modelcontextprotocol/servers>, and install it manually:
 - Open the Command Palette and search for "MCP: Add Server...":



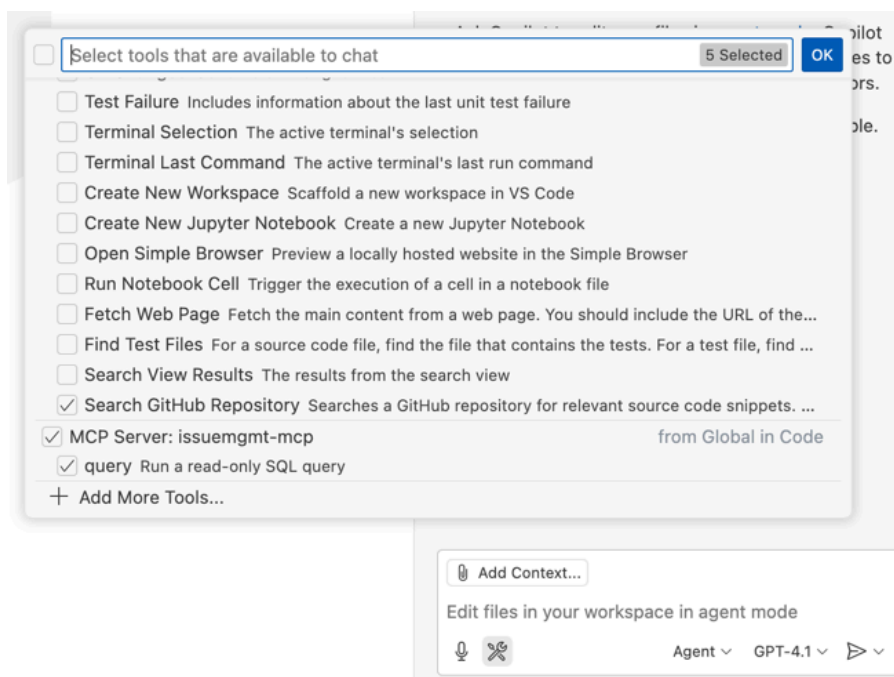
- Choose the correct MCP server type:



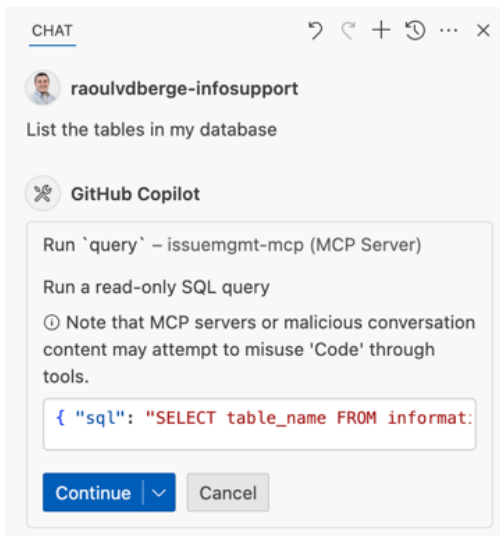
- Enter the details listed on the MCP server's project page, and other necessary configuration parameters of the MCP server itself.
6. Verify that the MCP server is running using the Command Palette > "MCP: List Servers...":



Additionally, verify that the tools added by the MCP server are listed under the tool menu in the chat window:



7. Send a prompt and trigger the MCP tool. It should infer that you want to run the tool, based on your prompt. If not, use the tool explicitly with #.



Triggering the PostgreSQL MCP query tool in agent mode

8. If the MCP tool is not triggered consistently, modify your `copilot-instructions.md` file and instruct Copilot to use your tool if your prompt wants it.



Bonus ideas

- Find an MCP server that provides reusable prompts, and use them from VS Code.
- Find an MCP server that provides resources, and use them from VS Code.



Lab 2 - Customization

Let's customize GitHub Copilot behavior using prompt files and instruction files to standardize and automate common development tasks.

- **Instruction files** influence how Copilot generates code and answers across all interactions within a project. They act as persistent behavioral guidelines.
- **Prompt files** are reusable, parameterized tasks you run as slash commands in Copilot Chat. They are contextual, not always active, and typically operate in agent mode.



Exercise 1: Coding standards in your project with an instruction file

Objective

Tune the suggestions and code generation from Copilot Chat to our project.

Instructions

1. Ensure that `github.copilot.chat.codeGeneration.useInstructionFiles` is set to `true` in your VS Code settings.
2. Create an instructions file. Choose between:
 - `.github/copilot-instructions.md` for project-wide instructions
 - `.github/instructions/*.instructions.md` for instructions to specific files (use `applyTo`)
3. In the instructions file, keep in mind that each instruction should be a single, simple statement.
4. Ask Copilot Chat to generate some code.

Examples of instructions

Add one statement per line, such as:

- Use `snake_case` for all variable names.
- Use the repository's existing `utils/` folder for shared code.
- Follow Clean Architecture principles for organizing new modules.
- Add input validation to all public functions.
- Write all tests using the `pytest` framework.

You can group instructions together by using a Markdown heading.

Observe

- How does the instruction file influence code generation?



Exercise 2: Standardizing AI interactions with prompt files

Objective

Use Copilot custom prompts to create a reusable template for common tasks.

Instructions

1. Identify a task in your codebase that has to be run often.
2. Create a prompt file `.github/prompts/*.prompt.md`.
3. Specify the chat mode, allowed tools, and description in the metadata.
4. Write the body of the prompt file.
 - You may use tools (like `#githubRepo`) or references to other files to provide more context.
5. Save and run the prompt file with a slash command.
6. Evaluate the result and refine your prompt if necessary.
 - **Tip:** Ask Copilot feedback on your prompt file.

Examples of custom prompts

- Onboarding prompt for new developers
- Generate API mocks
- Generate a new React form component (note how there is a lot of project-specific knowledge embedded in this prompt!):

```

---
mode: 'agent'
tools: ['githubRepo', 'codebase']
description: 'Generate a new React form component'
---

```

Your goal is to generate a new React form component based on the templates in #githubRepo contoso/react-templates.

Ask for the form name and fields if not provided.

Requirements for the form:

- * Use form design system components: [design-system/Form.md](../docs/design-system/Form.md)
- * Use `react-hook-form` for form state management
- * Always define TypeScript types for your form data
- * Prefer **uncontrolled** components using register
- * Use `defaultValues` to prevent unnecessary rerenders
- * Use `yup` for validation:
- * Create reusable validation schemas in separate files
- * Use TypeScript types to ensure type safety
- * Customize UX-friendly validation rules



Exercise 3: Custom chat participants

Objective

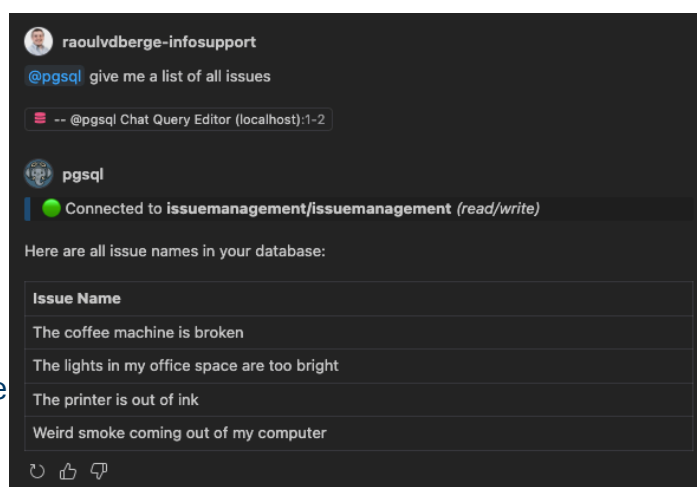
Let's use chat participants from an extension to provide a specialized view into a system, in ask mode.

Instructions

1. Go to the Visual Studio Code marketplace. Find extensions that add a custom chat participant using tag:chat-participant.
2. Install the extension and configure it so that you can use the chat participant.
3. Switch to Ask mode and try out the chat participant with @.

For example, for the PostgreSQL extension:

"@pgsql show table relationships for customer schema"





03. Proposal

Welcome to Smokefree Generation NL. The advocacy group for a smoke-free society in the Netherlands. We're working towards a proposal, so we haven't won the contract yet.

You've just missed our conversation with the business stakeholders. Luckily, we've recorded the meeting and have it transcribed. Our analyst also took 20 minutes to write down some key points in "03-input/assignment.md".

In short:

- Smokeless Generation NL maintains website and monthly newsletter "Opgelucht" (50,000 circulation)
- They want AI help to generate this newsletter.
- Input: Google Alerts, Professional reports and project updates
- To Do:
 - Automate reading Google Alerts via RSS feeds
 - Maintenance screen for managing RSS feeds
 - Bypass paywalls using archive services
 - Display news items for user selection
 - Auto-categorize (Government, Politics, Science, Education, etc.)
 - Auto-classify as Domestic/International
 - Generate articles following standard pattern (title, intro, summary, sources)
 - Publish drafts directly to Joomla CMS
 - Allow manual editing and final publication process

For our proposal, a mock-up is generated using <https://v0.app>. This is provided with the labs. Download the mockup images (taking screenshots is fine) and save the code.



Exercise 1: Make a proposal

Use the mockups to generate a proposal with the Copilot or ChatGPT.



Exercise 2: Function point analysis (functiepuntenanalyse) -- if time permits

Since we now have mockups for the app, we can use this as input for a function point analysis. You can do this by uploading the files in ChatGPT and ask for the analysis together with the assignment text.



04. Design

Congratulations 🎉! Your proposal has been accepted!

We can now start implementing the project. Before we can start writing down our specs, its a good idea to think up an architecture and design.

[!NOTE] *Before proceeding, open vscode in a new empty directory. This will be your working directory for this lab. You can `git init` it and commit your work as you go along.*



Exercise 1: Non-functional requirements

Time for some non-functional requirements. Often times non-functional requirements are not explicitly mentioned in the assignment. That's when a discussion with a Chat bot can help. Start a discussion about non-functional requirements by uploading your proposal and asking what non-functional requirements would be important or typical for a project of this scale. Keep in mind that whatever you come up with, you also need to deliver on. So don't go overboard.

After you have a preliminary list of non-functional requirements, let them be reviewed by the product-owner-by-proxy (the instructor). Once finalized, save the list as bullet points in a markdown document in docs/non-functional-requirements.md.

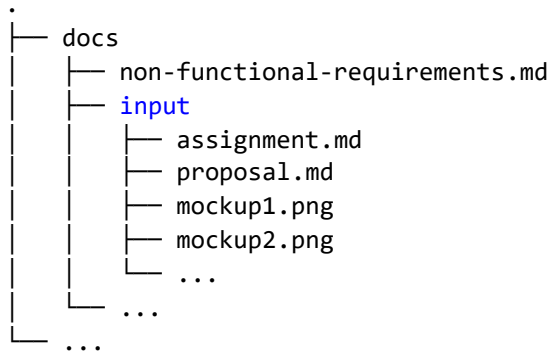


Exercise 2: Functional requirements

[!TIP] *From now on, we will use GitHub Copilot or similar AI tool alternately in ask and agent mode. Any output we create will be ready to check into source control.*

Once we have a finalized list of non-functional requirements, we can start writing down the functional requirements. Be sure all documents providing context are available to the AI tool you are using, including the assignment, proposal, mockups and non-functional requirements.

This is an example directory structure:



First, start in ask-mode to get a preliminary list of functional requirements. When you are happy, let it write them to `docs/functional-requirements.md`.

When you're happy with the list, ask it to expand each requirement in a feature file in `docs/features/[requirement-name].md`. Be sure to prompt it for a description, INVEST analysis, a list of user stories and acceptance criteria. Try to keep the list manageable for this training (around 10-15 features). If needed, ask it to prioritize and trim the list, we won't be implementing all features today.



Exercise 3: Minimal architectural design document

Our architect didn't have a lot of time, but did manage to scribble down a quick architectural design document. Copy `labs/04-design/architecture-design.md` to `/docs/architecture-design.md`.



Exercise 4: Additional architecture design -- If time permits

We can use the following prompt to help us write the architecture document. You can use this example prompt:

Generate an arc42 architecture document for the following non-functional requirements, architectural design and functional requirements. The functional requirements are written in "`docs/functional-requirements.md`", the non-functional requirements are written in "`docs/non-functional-requirements.md`", and the architectural design is written in "`docs/architecture-design.md`". I want the output to be saved in `docs/architecture` like:

- `01-introduction-goals.md`
- `02-architecture-overview.md`
- etc.

Exercise 5: Review all generated document -- if time permits

In real life, you would now review all design document with the team. This is the basis for your project, after all. For the this training you can look through it, or ask a different AI model to review them for you.



05 - Specifications



Prerequisites

Copy over the `.github/` directory from `labs/03-specs/` to your repository. This contains the issue templates and prompts we'll be using in this and following labs.

If you have followed the previous labs, you should have the following documents in your repository:

```
.
├── .github
│   ├── ISSUE_TEMPLATE
│   │   ├── feature.md
│   │   └── story.md
│   └── prompts
│       ├── backlog-feature.prompt.md
│       ├── backlog-story.prompt.md
│       ├── generate-plan.prompt.md
│       └── implement-plan.prompt.md
├── docs
│   ├── non-functional-requirements.md
│   ├── functional-requirements.md
│   ├── features
│   │   ├── feature1.md
│   │   ├── feature2.md
│   │   └── ...
│   ├── architecture (optional)
│   │   ├── 01-introduction-goals.md
│   │   ├── 02-architecture-overview.md
│   │   └── ...
│   └── input
│       ├── assignment.md
│       ├── proposal.md
│       ├── mockup1.png
│       ├── mockup2.png
│       └── ...
└── ...
```

[!TIP] Be sure to commit the code before proceeding.

Log into GitHub with your personal or work account. Be sure you have the GitHub CLI installed and are logged in. Create a new private repository for this lab and push the code you currently have:

```
# Create the repo:
gh repo create <your-github-username>/SmokeFreeGenerationNLContentPipeline --private

# Push the code:
git remote add origin https://github.com/<your-github-username>/SmokeFreeGenerationNLContentPipeline.git
# Or, for ssh:
# git remote add origin git@github.com:<your-github-username>/SmokeFreeGenerationNLContentPipeline.git
git branch -M main
git push --set-upstream origin main
```

To verify that the GitHub MCP extension is working, open the chat extension and ask: "Can you list all the issues in this github repository?". It should respond with something like "There are no issues in this repository", after consulting the GitHub MCP.



Exercise 1: Configure the backlog

We're going to organize the backlog for this project using GitHub issues. This step could be done by AI, but you don't need to use AI for things that can be scripted easily. We will be creating features, stories and bugs.

```
gh label create feature --color CC317C --description "Large body of work that delivers a strategic goal"
gh label create story --color 0366D6 --description "Distinct functionality that delivers value to users"
gh label create bug --color D73A4A --description "Something isn't working"
```



Exercise 2: Generate a feature

In this exercise, we will create specifications for the features we have identified as GitHub issues. We will use the GitHub issue template for features to create the specifications.

Open an important feature file in docs/features/. For example, adding an RSS feed or , open the chat in agent mode and type: /backlog-feature (or ask it to follow the instructions in .github/prompts/backlog-feature.prompt.md).

Verify the result. If necessary, edit the backlog-feature.prompt.md, remove the issue and try again. It is important to get this right, as we'll be using this as the basis for our implementation.

Next up: create the user stories for this feature. Open the chat in agent mode and type: /backlog-story <issue-number>, where <issue-number> is the issue number of the feature you just created.

Exercise 3: Generate the next feature -- If time permits

Repeat exercise 2 for the next feature in `docs/features/`. Note that, in practice, you would not want to do this for all features at once, but rather do this in iterations, implementing one feature at a time.



Lab 6: Implementation

It's finally time to implement the application. In this lab, we will use GitHub Copilot to help us implement the application based on the architecture and design we have created in the previous labs.



Preparations

To start, we will be creating a new nextjs application from scratch. This will be the basis for our implementation.

```
npm create next-app@latest content-pipeline  
# Choose "Yes, use recommended defaults"
```

It will generate a new project inside "content-pipeline" and install the dependencies. Next, we will copy that code into the root of the project.

```
mv content-pipeline/* .  
mv content-pipeline/./* .  
rmdir content-pipeline
```

Next, try it out with `npm run dev`.

[!TIP] Be sure to commit the code before proceeding.



Exercise 1: Create an implementation plan based on a user story

Pick one of your user stories from your issue tracker (be sure to not choose a feature itself). First, ask GitHub Copilot to create an implementation plan for this story, based on the architecture and design you have created in the previous labs.

See `inspect/generate-plan.prompt.md`, which is the prompt we use to create the implementation plan. You can also use it for inspiration on your projects, be sure to adapt it to your specific needs.

Some tips for creating good implementation plans:

- In the architecture documentation, there should be quality requirements about testing, data recovery, etc.
- Also add the scope: also what NOT to do, since the AI is too diligent here.
- Stop at one story, even though it could be tempting to implement too much here.

- Work via pull requests, so you can see the diff.

```
/generate-plan 42
```

Where 42 is the issue number of your user story. The AI will read the issue from GitHub and create an implementation plan as a comment to the issue.



Exercise 2: Implement a user story based on the implementation plan

After carefully reading the implementation plan, ask GitHub Copilot to implement the user story based on the implementation plan. We'll be using the prompt in `implement-plan.prompt.md` for this.

```
/implement-plan 42
```

Keep iterating until the implementation is complete. You can check the progress in the comments of the issue.



Exercise 3: Deploy to vercel -- If time permits

Ask GitHub Copilot to create a vercel configuration file and deploy the application to vercel.